

MTH209: Worksheet 5

Other Dimension Reduction Methods

Last few weeks we discussed Principal Component Analysis, which try to identify a lower dimensional linear subspace that accurately summarizes enough variability in the data. This is done by finding the largest k leading eigenvectors of the covariance matrix, Σ . However, this dimension reduction is only linear and not suitable for situations when reduction is most fruitful with a change in geometry. Let's see what we mean by comparing with other methods.

t-SNE

The t-distributed Stochastic Neighbor Embedding (t-SNE) method is a nonlinear dimensionality reduction technique. t-SNE does not attempt to preserve global geometric structure, such as distances or variances, but instead focuses on preserving local neighborhood relationships by modeling distances using a t distribution.

The starting point of t-SNE is the idea that high-dimensional data can be described through conditional similarity distributions. Given observations $x_1, \dots, x_n \in \mathbb{R}^d$ in a high-dimensional space, t-SNE defines, for each observation x_i , a conditional probability distribution over all other points. This probability distribution is determined by the distance of all the points from i . This distribution reflects how likely x_j would be selected as a neighbor of x_i . These probabilities are defined using a Gaussian kernel centered at x_i , given by (for $i \neq j$)

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / (2\sigma_i^2))}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / (2\sigma_i^2))}.$$

The denominator ensures that these $p_{j|i}$ describe a valid probability mass function over all x_j . Note that $p_{i|i} = 0$ is assumed. The parameter σ_i^2 is chosen differently for each observation i and is chosen to ensure a desirable number of “neighbors” are given reasonable probability. That is, if σ_i^2 is too small, then only the points very close to x_i are given high $p_{j|i}$ values. Larger value of σ_i^2 distributes the distribution to larger number of neighbors.

Note now that it is possible for $p_{j|i}$ to be different from $p_{i|j}$ and for this reason the following is done. To obtain a symmetric joint probability model, t-SNE defines

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n},$$

which can be interpreted as a probability mass function over pairs of observations. If i is “close” to j and j is

“close” to i , p_{ij} is large, indicating a high affinity of this pair. If p_{ij} is small for a pair, it means that either neither are close to each other, or one of them is somewhat close and in the other direction they are not close. These probabilities satisfy $\sum_{i \neq j} p_{ij} = 1$, allowing the high-dimensional structure to be viewed as a discrete probability distribution over pairs.

Now, the goal is to do dimension reduction. So we want to find points $y_1, \dots, y_n \in \mathbb{R}^k$ where each y_i corresponds to an x_i . These y_i are called *embeddings*. This will be done by creating a similarly motivated distribution on the unknown y_i s and then finding the best possible set of y_i s that minimize the distance between p_{ij} and the new distribution.

In the low-dimensional embedding space, with points $y_1, \dots, y_n$ typically in two or three dimensions, t-SNE defines an analogous probability distribution q_{ij} . Instead of Gaussian kernels, it uses a heavy-tailed Student t distribution with one degree of freedom, which corresponds to a Cauchy distribution. The low-dimensional similarities are therefore defined as

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}.$$

The statistical motivation for this choice is closely related to the so-called crowding problem. When mapping from high to low dimensions, moderate pairwise distances in the original space cannot all be represented faithfully using the Gaussian type method used in p_{ij} since this leads to large clustering and “squishing” of the points in the y space. The heavy tails of the t distribution allow moderately distant points in the original space to not cluster in the reduced space.

The central objective function of t-SNE is the Kullback–Leibler divergence between the high-dimensional and low-dimensional pairwise distributions. Specifically, the embedding is obtained by minimizing

$$\text{KL}(P|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

That is, the objective is to find

$$\arg \min_{y_1, y_2, \dots, y_n \in \mathbb{R}^k} \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

This minimization, unfortunately, does not have a closed form solution and iterated algorithms are required to obtain the minima. Further, the function being minimized $\text{KL}(P|Q)$ over these sets of observations is *highly non-convex* which implies that any algorithms that attempts to find a minima will often find local minimas instead of global minimas. The optimization is typically performed using gradient descent with momentum. The coordinates are initialized using small random values, and the objective function is minimized iteratively.

Finally, it is important to note that t-SNE provides no consistent estimator of an underlying population parameter. The embedding is best understood as an *optimization-based visualization* derived from finite-sample empirical distributions. In this sense, t-SNE is fundamentally descriptive rather than inferential. That is, t-SNE cannot be used for predictions since new observations cannot be mapped into this new space (unlike PCA).

Choosing σ_i

Providing a different value of σ_i for each observation will be unreasonable, so instead the algorithm fixes a desired *perplexity*. Then a σ_i value is set by the algorithm ensuring that perplexity is received.

$$\log_2(\text{Perplexity}) = - \sum_{i \neq j} p_{j|i} \log_2 p_{j|i}.$$

A small value of perplexity yields a small σ_i encouraging smaller clustering based on very local behavior, and a large value of perplexity encourages global features being captured. A perplexity of 5-50 is recommended.

UMAP

UMAP (Uniform Manifold Approximation and Projection) is another nonlinear dimension reduction. It has similar fundamentals to t-SNE however

- it creates p_{ij} differently, and p_{ij} do not create a pmf over all possible pairs. Instead p_{ij} just represent probabilities.
- it defines q_{ij} also as probabilities between the unknown embeddings $y_1, y_2, \dots, y_n \in \mathbb{R}^k$.
- A similar KL distance is minimized to find the y s.
- In order to implement UMAP, we need to provide it the number of neighbors, similar to perplexity.

Disadvantages

Both tSNE and UMAP are primarily used for visualizations. Beyond that they are not very useful (UMAP may be more useful than tSNE). PCA on the other hand can be useful for predictions as we can store the eigenvectors to be used in the future as well.

Reference

Theoretical reference:

Wang, Yingfan, et al. "Understanding how dimension reduction tools work: an empirical approach to deciphering t-SNE, UMAP, TriMAP, and PaCMAP for data visualization." *Journal of Machine Learning Research* 22.201 (2021): 1-73.

Practical: <https://bioinformatics-core-shared-training.github.io/CrukCiScRnaSeqMatDev/AnaWiSce/AnaCourse1/BookDownCourse1/dimRedForVizTop.html>

Questions

1. Run the code in `tsne.R` file and understand the t-SNE method.
 - a. What happens to the $p_{j|i}$ plot when you set perplexity to be 50?
 - b. Rerun the code increasing $n = 1000$ and testing for different perplexities.

2. Implement 2-dimensional t-SNE for the iris dataset and compare the visualizations of the 2-dimensional mapping with PCA. Play around with perplexity values until you get a visualization that looks nice.
3. Install the library `uwot` and use the function `umap` to run UMAP on the iris dataset.
4. Implement PCA, UMAP, tSNE on the wine dataset given. More information on the dataset is here: <https://archive.ics.uci.edu/dataset/109/wine>
Which of the three methods is best able to separate the three types of wine?
5. The IMIFA R package has the dataset on hand written numbers in `data(USPSdigits)`.
 - a. Understand the dataset first by recreating the first 6 images in the dataset.
 - b. Make a subset of 2000 images of the dataset (otherwise it will take too long)
 - c. Implement PCA on the dataset and plot the two principal components marked with the true label.
 - d. Implement t-SNE with and plot the embeddings with the labels marked.
 - e. Implement UMAP and plot the embeddings with the labels marked.
 - f. Find an observation in the UMAP embedding space that seems to lie in a wrong cluster. View the original image and try to see why the algorithm got it wrong.